

Package ‘geomattR’

June 19, 2026

Type Package

Title Calculate Geometric Attributes of Spatial Polygons

Version 0.1.0

Description Provides a comprehensive toolkit for calculating geometric and morphometric attributes of spatial polygons. Computes metrics including area, perimeter, compactness, elongation, orientation (bearing), fractal dimension, and various shape indices. All measurements use geodesic calculations for accuracy across different coordinate reference systems, automatically handling geographic (lon/lat) data appropriately. Designed for geospatial analysts, urban planners, and environmental scientists working with 'terra' vector data objects <<https://github.com/rspatial/terra>>.

License MIT + file LICENSE

URL <https://github.com/gortegasolis/geomattR>

BugReports <https://github.com/gortegasolis/geomattR/issues>

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Imports terra (>= 1.8.70),
geosphere (>= 1.5.20),
methods

Suggests testthat (>= 3.2.3),
knitr (>= 1.50),
rmarkdown (>= 2.30)

Config/testthat/edition 3

VignetteBuilder knitr

Contents

calculate_geometric_attributes	2
calculate_geometric_attributes_single	4
calc_elongation	6
calc_extent_ew	6
calc_extent_ns	7
get_distant_points	7

Index	9
--------------	----------

`calculate_geometric_attributes`
Calculate Geometric Attributes of Spatial Polygons

Description

Calculates a comprehensive set of geometric and morphometric attributes for spatial polygon features. Supports both sequential and parallel execution through an optional cluster argument.

Usage

```
calculate_geometric_attributes(v, metrics = "all", cl = NULL)
```

Arguments

- | | |
|----------------------|---|
| <code>v</code> | A <code>SpatVector</code> object representing one or more polygons. |
| <code>metrics</code> | Character string or vector specifying which metrics to calculate. Options are: <ul style="list-style-type: none"> • "all" (default): Calculate all available metrics • A single metric name as a string: Calculate only that metric • A character vector of metric names: Calculate specified metrics Available metric names: "area", "perimeter", "compactness", "reock", "elongation_rectangle", "num_holes", "hole_area", "hole_area_pct", "num_polygons", "ew_length", "ns_length", "maxlength", "bearing", "northernness", "fractaldimension", "sinuosity", "shape_index", "circularity_ratio", "decimallongitude", "decimallatitude" |
| <code>cl</code> | A cluster object created by makeCluster , or <code>NULL</code> (default). When <code>NULL</code> features are processed sequentially. Passing a cluster enables parallel processing across the cluster workers. |

Details

Each polygon is processed independently via [calculate_geometric_attributes_single](#), which ensures that intermediate objects (convex hull, minimum circle, etc.) are computed only once per feature.

Sequential vs. Parallel:

- When `cl = NULL` (default), features are processed sequentially with [lapply](#).
- When `cl` is a valid cluster, features are distributed across workers via [parLapply](#). The function handles library loading and function export to the cluster automatically.

Geodesic Measurements: All measurements use geodesic calculations for accuracy regardless of input CRS. Inputs are internally projected to EPSG:4326 for computation and restored to the original CRS on return.

Size metrics:

- Area: `terra::expanses()` with `transform = TRUE`.
- Perimeter: `terra::perim()` on the geographic representation.
- Extents (EW, NS): geodesic distances between bounding-box corners.

Shape metrics:

- Compactness (Polsby-Popper): how circular the polygon is.
- Shape index and circularity ratio: alternative shape characterizations.
- Fractal dimension: boundary complexity.

Orientation metrics:

- Bearing: orientation of the longest axis.
- Northerness: cosine of bearing; ranges from -1 (south) to 1 (north).
- Elongation: ratio of major to minor axis of the minimum bounding rectangle.

Value

The input `SpatVector` with additional columns containing the requested geometric attributes:

- `area`: Total area in square meters
- `perimeter`: Total perimeter length in meters
- `compactness`: Polsby-Popper compactness (1 = perfect circle)
- `reock`: Reock compactness (area / minimum enclosing circle area)
- `elongation_rectangle`: Elongation ratio based on minimum bounding rectangle
- `num_holes`: Number of holes (interior rings) in the polygon
- `hole_area`: Total area of holes in square meters
- `hole_area_pct`: Percentage of total area occupied by holes
- `num_polygons`: Number of separate polygon parts (multi-part count)
- `ew_length`: Average east-west extent in meters
- `ns_length`: Average north-south extent in meters
- `maxlength`: Maximum distance across convex hull in meters
- `bearing`: Geographic bearing of maximum length line in degrees
- `northerness`: Northerness component of bearing (-1 to 1)
- `fractaldimension`: Fractal dimension (complexity measure, typically 1-2)
- `sinuosity`: Sinuosity (perimeter to maximum length ratio)
- `shape_index`: Shape index (deviation from circular shape)
- `circularity_ratio`: Circularity based on maximum length
- `decimallongitude`: Centroid longitude in decimal degrees
- `decimallatitude`: Centroid latitude in decimal degrees

Examples

```
library(terra)

# Create a sample 1-degree square polygon near the equator
coords <- cbind(c(0, 0, 1, 1, 0), c(0, 1, 1, 0, 0))
polygons <- vect(coords, type = "polygon", crs = "EPSG:4326")

# Sequential processing (default)
result <- calculate_geometric_attributes(polygons)
```

```
# Calculate specific metrics only
result <- calculate_geometric_attributes(polygons,
  metrics = c("area", "perimeter", "compactness"))

## Not run:
# Parallel processing with an explicit cluster (use small number of cores for checks)
cl <- parallel::makeCluster(2)
result <- calculate_geometric_attributes(polygons, cl = cl)
parallel::stopCluster(cl)

## End(Not run)
```

calculate_geometric_attributes_single

Calculate Geometric Attributes for a Single Polygon

Description

Calculates geometric attributes for exactly one polygon feature. This is the core computational function used internally by [calculate_geometric_attributes](#).

Usage

```
calculate_geometric_attributes_single(v, metrics = "all")
```

Arguments

v	A SpatVector object containing exactly one polygon feature.
metrics	Character string or vector specifying which metrics to calculate. Options are: • "all" (default): Calculate all available metrics • A single metric name as a string: Calculate only that metric • A character vector of metric names: Calculate specified metrics Available metric names: "area", "perimeter", "compactness", "reock", "elongation_rectangle", "num_holes", "hole_area", "hole_area_pct", "num_polygons", "ew_length", "ns_length", "maxlength", "bearing", "northernness", "fractaldimension", "sinuosity", "shape_index", "circularity_ratio", "decimallongitude", "decimallatitude" (see Details for descriptions).

Details

This function is optimized to process a single polygon. It computes intermediate geometric objects (convex hull, centroid, etc.) only once and reuses them across dependent metrics, avoiding redundant computation.

Metric dependencies are resolved automatically: requesting a derived metric (e.g., "sinuosity") will internally compute its prerequisites (perimeter, maxlength) even if they are not explicitly requested.

Geodesic Measurements: All measurements use geodesic calculations for accuracy across different coordinate systems. The input is internally projected to EPSG:4326 for computation and restored to its original CRS before returning.

- **area:** Total area in square meters (geodesic via `terra::expanse()`).

- **perimeter**: Total perimeter in meters (geodesic via `terra::perim()`).
- **compactness**: Polsby-Popper formula: $(4\pi \cdot A)/P^2$. Values 0–1; 1 = circle.
- **reock**: Ratio of polygon area to minimum enclosing circle area.
- **elongation_rectangle**: Major/minor axis ratio of the minimum bounding rectangle.
- **num_holes**: Count of interior rings (holes).
- **hole_area**: Total hole area in square meters.
- **hole_area_pct**: Percentage of total area occupied by holes.
- **num_polygons**: Number of parts in a multi-part polygon.
- **ew_length**: Average east-west extent in meters (geodesic).
- **ns_length**: Average north-south extent in meters (geodesic).
- **maxlength**: Maximum geodesic distance across the convex hull.
- **bearing**: Bearing (degrees) from southernmost to northernmost hull vertex.
- **northernness**: $\cos(\text{bearing} \cdot \pi/180)$.
- **fractaldimension**: $2 \log(P)/\log(A)$.
- **sinuosity**: Perimeter / maxlength.
- **shape_index**: $P/(2\sqrt{\pi A})$.
- **circularity_ratio**: $4A/(\pi \cdot L^2)$ where $L = \text{maxlength}$.
- **decimallongitude**: Centroid longitude (decimal degrees).
- **decimallatitude**: Centroid latitude (decimal degrees).

For processing multiple polygons use [calculate_geometric_attributes](#) which wraps this function and supports both sequential and parallel execution.

Value

The input `SpatVector` with additional columns containing the requested geometric attributes.

Examples

```
library(terra)

# Create a sample 1-degree square polygon near the equator
coords <- cbind(c(0, 0, 1, 1, 0), c(0, 1, 1, 0, 0))
polygon <- vect(coords, type = "polygon", crs = "EPSG:4326")

# Calculate all attributes
result <- calculate_geometric_attributes_single(polygon)

# Calculate only specific metrics
result <- calculate_geometric_attributes_single(polygon,
  metrics = c("area", "perimeter", "compactness"))
```

calc_elongation

Calculate Elongation Ratio from Minimum Bounding Rectangle

Description

Calculates the elongation ratio of a polygon based on its minimum bounding rectangle. The ratio is the major axis length divided by the minor axis length. The minimum bounding rectangle is computed from the convex hull of the input polygon. It is important to notice that this method assumes the minimum bounding rectangle is unique, which may not always be the case. Consider this with caution.

Usage

```
calc_elongation(v, hull = NULL)
```

Arguments

v	A SpatVector object representing a polygon.
hull	An optional pre-computed convex hull (SpatVector). If NULL (default), the convex hull is computed internally. Passing a pre-computed hull avoids redundant computation.

Value

A numeric value representing the elongation ratio (major/minor axis) of the input polygon. Higher values indicate more elongated shapes.

Examples

```
library(terra)
coords <- cbind(c(0, 0, 1, 1, 0), c(0, 1, 1, 0, 0))
polygon <- vect(coords, type = "polygon", crs = "EPSG:4326")
elongation <- calc_elongation(polygon)
```

calc_extent_ew

Calculate East-West Extent

Description

Calculates the average east-west extent of a polygon based on its geographic bounding box.

Usage

```
calc_extent_ew(v)
```

Arguments

v	A SpatVector object representing a polygon.
---	---

Value

A numeric value representing the average east-west extent in meters.

Examples

```
library(terra)
coords <- cbind(c(0, 0, 1, 1, 0), c(0, 1, 1, 0, 0))
polygon <- vect(coords, type = "polygon", crs = "EPSG:4326")
ew_extent <- calc_extent_ew(polygon)
```

calc_extent_ns	<i>Calculate North-South Extent</i>
----------------	-------------------------------------

Description

Calculates the average north-south extent of a polygon based on its geographic bounding box.

Usage

```
calc_extent_ns(v)
```

Arguments

v A SpatVector object representing a polygon.

Value

A numeric value representing the average north-south extent in meters.

Examples

```
library(terra)
coords <- cbind(c(0, 0, 1, 1, 0), c(0, 1, 1, 0, 0))
polygon <- vect(coords, type = "polygon", crs = "EPSG:4326")
ns_extent <- calc_extent_ns(polygon)
```

get_distant_points	<i>Find largest distance between a pair of opposite vertices</i>
--------------------	--

Description

Identifies the pair of most distant points on a polygon's convex hull and calculates the geodesic distance and bearing from the southernmost to the northernmost point.

Usage

```
get_distant_points(v, hull = NULL, distance = TRUE, bearing = TRUE)
```

Arguments

<code>v</code>	A <code>SpatVector</code> object representing a polygon.
<code>hull</code>	An optional pre-computed convex hull (<code>SpatVector</code>). If <code>NULL</code> (default), the convex hull is computed internally. Passing a pre-computed hull avoids redundant computation when calling from higher-level functions.
<code>distance</code>	Logical indicating whether to return the maximum distance (default <code>TRUE</code>).
<code>bearing</code>	Logical indicating whether to return the bearing (default <code>TRUE</code>).

Details

The most distant hull-vertex pair is found with rotating calipers over the convex hull (linear in the number of hull vertices), then geodesic distances are evaluated only for candidate antipodal pairs.

Value

A list containing:

<code>south_point</code>	<code>SpatVector</code> of the southernmost point
<code>north_point</code>	<code>SpatVector</code> of the northernmost point
<code>distance</code>	Maximum distance in meters (if <code>distance = TRUE</code>)
<code>bearing</code>	Geographic bearing from south to north in degrees (if <code>bearing = TRUE</code>)

Examples

```
library(terra)
coords <- cbind(c(0, 0, 1, 1, 0), c(0, 1, 1, 0, 0))
polygon <- vect(coords, type = "polygon", crs = "EPSG:4326")
distant_pts <- get_distant_points(polygon)
```


Index

`calc_elongation`, [6](#)
`calc_extent_ew`, [6](#)
`calc_extent_ns`, [7](#)
`calculate_geometric_attributes`, [2](#), [4](#), [5](#)
`calculate_geometric_attributes_single`,
 [2](#), [4](#)

`get_distant_points`, [7](#)

`lapply`, [2](#)

`makeCluster`, [2](#)

`parLapply`, [2](#)